

Лабораторная работа №5. Использование основных средств технологии OpenMP.

Постановка задачи.

Дана последовательная программа, реализующая алгоритм релаксации Якоби. Требуется разработать параллельную программу с использованием технологии OpenMP и провести исследование ее эффективности.

Задание.

1. Реализовать и отладить последовательную версию программы
2. Разработать параллельную версию программы с использованием технологии OpenMP
3. Исследовать время выполнения разработанной программы в зависимости от размера сетки и количества используемых ядер.
4. Построить табл. 1. При построении таблицы принять k равным номеру по списку группы, а $k\%4$ равным остатку от деления k на 4.

Табл. 1.

N	itmax $k\%4$	Последовательный алгоритм, время с.	Параллельный алгоритм			
			1 ядро		2 ядра	
			Время с.	Ускорение	Время с.	Ускорение
128 + k	$k\%4$					
160 + k	$k\%4$					
192 + k	$k\%4$					
224 + k	$k\%4$					
256 + k	$k\%4$					

5. Построить 5 графиков зависимости времени выполнения последовательного алгоритма, параллельных алгоритмов и ускорений от N.

6. Обоснуйте выполнимость закона Амдала

Приложение. Последовательная программа, реализующая алгоритм релаксации Якоби.

```
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#define Max(a,b) ((a)>(b)?(a):(b))
#define N 768
double maxeps = 0.1e-7;
int itmax = 10;
int i,j,k;
double eps;
double A [N][N][N], B [N][N][N];

void relax();
void resid();
void init();
void verify();
void wtime();

int main(int an, char **as)
{
    int it;
    double time0, time1;
    init();
    /* time0=omp_get_wtime (); */
    wtime(&time0);
    for(it=1; it<=itmax; it++)
    {
        eps = 0.;
        relax();
        resid();
        printf( "it=%4i  eps=%f\n", it,eps);
        if (eps < maxeps) break;
    }
    wtime(&time1);
    /* time1=omp_get_wtime (); */
    printf("Time in seconds=%gs\t",time1-time0);
    verify();
    return 0;
}

void init()
{
```

```

    for(i=0; i<=N-1; i++)
        for(j=0; j<=N-1; j++)
            for(k=0; k<=N-1; k++)
                {
                    if(i==0 || i==N-1 || j==0 || j==N-1 || k==0 || k==N-1)
A[i][j][k]= 0.;
                    else A[i][j][k]= ( 4. + i + j + k );
                }
}

void relax()
{
    for(i=1; i<=N-2; i++)
        for(j=1; j<=N-2; j++)
            for(k=1; k<=N-2; k++)
                {
                    B[i][j][k]=(A[i-1][j][k]+A[i+1][j][k]+A[i][j-
1][k]+A[i][j+1][k]+A[i][j][k-1]+A[i][j][k+1])/6.;
                }
}

void resid()
{
    for(i=1; i<=N-2; i++)
        for(j=1; j<=N-2; j++)
            for(k=1; k<=N-2; k++)
                {
                    double e;
                    e = fabs(A[i][j][k] - B[i][j][k]);
                    A[i][j][k] = B[i][j][k];
                    eps = Max(eps,e);
                }
}

void verify()
{
    double s;
    s=0.;
    for(i=0; i<=N-1; i++)
        for(j=0; j<=N-1; j++)
            for(k=0; k<=N-1; k++)
                {
                    s=s+A[i][j][k]*(i+1)*(j+1)*(k+1)/(N*N*N);
                }
}

```

```
        printf(" S = %f\n",s);
    }

void wtime(double *t)
{
    static int sec = -1;
    struct timeval tv;
    gettimeofday(&tv, (void *)0);
    if (sec < 0) sec = tv.tv_sec;
    *t = (tv.tv_sec - sec) + 1.0e-6*tv.tv_usec;
}
```